

Media Mix Modelling (MMM)

Phase 0: Environment and Project Setup

Goal: clean repo, reproducible environment, project skeleton

Set up a GitHub repo with a logical folder structure from day one — something like `data/`, `notebooks/`, `src/`, `outputs/`, and `writeup/`. Use a virtual environment with pinned dependencies. Your core libraries will be PyMC, ArviZ (for diagnostics and plotting), NumPy, Pandas, Matplotlib/Seaborn, and SciPy for the optimization layer. Write a README stub immediately so you're forced to articulate what the project is from the start.

Deliverable: Repo is live, environment reproducible with a single command, folder structure reflects the five layers.

Phase 1: Synthetic Data Generation

Goal: a realistic dataset with known ground truth that you can use to validate the entire pipeline

This phase is more intellectually demanding than it sounds. You're not just generating random numbers — you're encoding your causal assumptions into a data-generating process.

Start by deciding your world: 3 regions, 4 marketing channels (e.g. TV, Paid Search, Social, Display), weekly data over 3 years. Then build the DGP in this order:

First, create the baseline demand signal — a trend component plus seasonality (use a Fourier series to model annual cycles, which also foreshadows what you'll do in the model). Add holiday spikes manually.

Second, generate raw spend per channel per region. Make it realistic — TV spend is lumpy and high variance, Search is steadier, Social has growth over time. Add a promotional calendar that creates spend bursts.

Third, apply your ground-truth Adstock and saturation transformations to each channel's spend. Store the true parameter values (decay rates, Hill slopes, half-saturation points) in a config file. These are your recovery targets.

Fourth, compute the true contribution of each channel as a linear combination of the transformed spend, with known coefficients per region. Add noise. Sum everything to get revenue.

Save both the raw data (what the model sees) and a ground truth table (channel contributions, true parameters) separately. The ground truth table becomes your evaluation benchmark throughout the project.

Deliverable: `data/synthetic_raw.csv` and `data/ground_truth.csv`. A notebook showing the DGP visually — spend patterns, seasonality, and what the true channel contributions look like.

Phase 2: Adstock and Saturation Transformations

Goal: implement the media transformations as standalone, tested functions before they go anywhere near a model

Build these in `src/transforms.py` as pure functions. You want three things:

Geometric Adstock: takes a spend vector and a decay rate α , returns the carryover-weighted series. The formula is a recursive convolution — each period's adstock is current spend plus α times the previous adstock. Implement it, then plot it for several values of α so you can see intuitively what it does. α near 0 means almost no carryover, α near 0.9 means the ad effect lingers for months.

Hill Saturation: takes a transformed spend vector plus a slope (k) and half-saturation (ec) parameter, returns a value bounded between 0 and 1 representing the saturation-adjusted effect. Plot the curve for several parameter combinations so you understand the shape — this is the S-curve / diminishing returns curve.

Weibull Adstock (optional but impressive): a more flexible two-parameter generalization of geometric decay. Add this if you want to show extra depth.

Crucially, write unit tests for these functions. Check edge cases: zero spend, very high decay rates, the asymptotic behavior of Hill at extreme inputs. Tested utility functions in a portfolio project are rare and signal engineering maturity.

Deliverable: `src/transforms.py` with documented functions, a test file, and a notebook visualizing the transformation behavior across parameter ranges.

Phase 3: Bayesian Model — Single Region First

Goal: get a working, interpretable model on one region before adding hierarchy

Always start simpler than you think you need to. Build the model for one region only. This keeps the PyMC code manageable and lets you establish the core structure cleanly.

The model structure is: Revenue = Baseline (trend + seasonality) + $\sum (\beta_{\text{channel}} \times \text{Hill}(\text{Adstock}(\text{spend}_{\text{channel}}))) + \text{noise}$.

Work through it in this order within PyMC:

Priors first. Before writing the likelihood, think carefully about each parameter. Decay rates live in $[0,1]$ — Beta distribution is natural. Hill slopes are positive — HalfNormal or Gamma. Channel coefficients are positive (more spend can't hurt revenue in expectation) — HalfNormal. Revenue noise — HalfNormal on σ . Write a short justification for each prior choice in a comment. This discipline pays off in the writeup.

Prior predictive checks. Sample from the prior only (`pm.sample_prior_predictive()`), then plot the implied revenue distribution against your actual revenue range. Does the model allow revenue of negative ten million? Revenue of ten billion? Tune your priors until the prior predictive is diffuse but sane. This step is the most commonly skipped and the most impressive to include.

Likelihood and fitting. Add the Normal likelihood, then run NUTS sampling. Start with a small number of samples to make sure it runs, then scale up.

Diagnostics. Use ArviZ for everything: R-hat (should be < 1.01 for all parameters), effective sample size, trace plots, divergence counts. If you have divergences, investigate — they usually mean your model geometry is problematic, often fixable by reparameterization (e.g. non-centered parameterization for hierarchical models).

Parameter recovery. Plot your posterior distributions against the true parameter values from your ground truth config. Do the true values fall within the posterior? This is your quantitative model validation and it's a killer result to show.

Deliverable: A notebook with the single-region model, prior predictive plots, posterior diagnostics, and a parameter recovery plot.

Phase 4: Hierarchical Extension

Goal: extend to all regions with partial pooling

Now you add the hierarchy. Each region gets its own parameters, but those parameters are drawn from shared population distributions. The practical PyMC pattern is: define hyperpriors (population mean μ and standard deviation σ for each parameter family), then define per-region parameters as offset from the population mean (non-centered parameterization is important here for sampler efficiency).

For example, the TV decay rate for region r is: $\alpha_r = \mu_\alpha + \sigma_\alpha \times z_r$, where $z_r \sim \text{Normal}(0,1)$. This is mathematically equivalent to $\alpha_r \sim \text{Normal}(\mu_\alpha, \sigma_\alpha)$ but samples much better in NUTS.

After fitting, you should be able to show two interesting things. First, the pooling effect — regions with less data have posteriors closer to the population mean than regions with more data. Second, the population-level posteriors — what does a "typical" region look like? This is the business-level insight.

Re-run your parameter recovery analysis for all regions. Also check that the model correctly identifies that data-sparse regions have wider posteriors.

Deliverable: Extended notebook or new notebook with hierarchical model, pooling visualization, and multi-region parameter recovery.

Phase 5: Posterior Analysis and Channel Attribution

Goal: translate model outputs into business-readable insights

This is where you compute the things stakeholders actually want. Using your posterior samples:

Channel contribution decomposition: For each posterior sample, compute the predicted revenue contribution of each channel (the $\beta \times \text{Hill}(\text{Adstock}(\text{spend}))$ term) as a fraction of total predicted revenue. Average over samples to get mean contribution with credible intervals. Plot this as a stacked area chart over time and as a bar chart of total contribution per channel. You now have probabilistic attribution.

Response curves: Plot the posterior mean (and 90% credible band) of the Hill saturation curve for each channel. This shows the current operating point on the curve — are you in the diminishing returns zone? This directly motivates the optimization step.

Return on Ad Spend (ROAS): Compute the marginal ROAS for each channel at current spend levels — this is the derivative of the response curve at current spend, which you can compute analytically from the Hill function parameters.

Deliverable: Attribution decomposition plots, response curve plots, ROAS table with uncertainty, all in a clean analysis notebook.

Phase 6: Budget Optimization

Goal: turn the model into a decision tool

Frame this as a constrained optimization: maximize expected revenue subject to total budget = current total budget (or a range of budget scenarios).

The approach is to take a large number of posterior samples, and for each sample, run a numerical optimizer (SciPy's `minimize` with the SLSQP method handles bound and equality constraints well) over the budget allocation vector, using the posterior sample's parameters to define the response curves. Collect the optimal allocations across samples to get a distribution over optimal budgets.

The output is: for each channel, the current spend, the posterior mean optimal spend, and the 90% credible interval on optimal spend. This is the business recommendation with honest uncertainty.

Also run this across a range of total budget levels (budget response curves) — what happens to optimal revenue as total budget increases? This is the kind of analysis a real marketing team would ask for.

Deliverable: Optimization notebook with current vs. optimal allocation table, uncertainty on recommendations, and budget response curves.

Phase 7: Writeup and Presentation

Goal: a document that stands alone and tells the full story

Structure the writeup as a long-form technical post, roughly 3,000–4,000 words. The arc should be:

Introduction — why MMM, why now, why Bayesian. **Causal framing** — your DAG, what assumptions you're making and why they matter. **Model design** — walk through every component and justify every choice. **Validation** — prior predictive checks, MCMC diagnostics, parameter recovery. **Results** — attribution, response curves, ROAS. **Optimization** — the allocation recommendation and what it means. **Limitations and future work** — be honest about what the model can't do (it can't handle unobserved confounders, the causal claims require untestable assumptions, etc.). This section builds credibility rather than undermining it.

Polish the figures — consistent color scheme, labeled axes, confidence bands everywhere. The writeup should be readable by a technical hiring manager in 20 minutes and leave them with no doubt that you understand what you built.

Deliverable: [writeup/mmm_writeup.md](#) or a Medium/personal site post. Clean README linking to the writeup, the key notebooks, and the key results.